

# DrFresh

Ouzas Vasileios   Alino Papagiannis

May 12, 2026

---

ECE490 – IoT System Architecture Proposal

University of Thessaly

## Contents

---

|           |                                      |           |
|-----------|--------------------------------------|-----------|
| <b>1</b>  | <b>System Overview</b>               | <b>2</b>  |
| <b>2</b>  | <b>Hardware Architecture</b>         | <b>2</b>  |
| <b>3</b>  | <b>Data Flow &amp; Communication</b> | <b>2</b>  |
| <b>4</b>  | <b>Data Modeling</b>                 | <b>4</b>  |
| <b>5</b>  | <b>Data Storage Strategy</b>         | <b>5</b>  |
| <b>6</b>  | <b>Processing &amp; System Logic</b> | <b>6</b>  |
| <b>7</b>  | <b>Reliability Considerations</b>    | <b>7</b>  |
| <b>8</b>  | <b>Data Visualization</b>            | <b>8</b>  |
| <b>9</b>  | <b>Design Justification</b>          | <b>9</b>  |
| <b>10</b> | <b>Implementation Reality Check</b>  | <b>10</b> |

# 1 System Overview

---

DrFresh is a smart liquid refreshment dispenser designed to automate and monitor the serving of beverages from two separate tanks. The system addresses the need for controlled, trackable, and user-friendly liquid dispensing in environments such as offices or small businesses, where both convenience and usage monitoring are important.

The system is built around a Raspberry Pi acting as the central controller. It manages two water pumps through a dual-channel relay module, handles all communication via the MQTT protocol, and maintains operational state in a local SQLite database. Dispensing events, alerts, tank volume snapshots, and computed analytics are recorded in InfluxDB and visualized through a Grafana dashboard. The system supports two dispensing modes: an automatic mode that dispenses a fixed default volume, and a manual mode where the user controls the duration of dispensing by sending continuous heartbeat messages. Safety mechanisms ensure pumps are never left running unintentionally, even under network or user error conditions.

## 2 Hardware Architecture

---

The hardware architecture revolves around the Raspberry Pi as the main control and processing unit. It combines GPIO hardware control, networking, local storage, and application logic in a single device, which simplifies the overall system design.

A dual-channel relay module is connected to two GPIO pins on the Raspberry Pi (BCM pins 17 and 27 for pumps A and B respectively). The relay module acts as a safe electrical switch, allowing the low-voltage GPIO signals from the Pi to control the 6–12 V water pumps without direct electrical coupling. Both pumps are capable of a maximum flow rate of 3 L/min and are connected to their respective liquid tanks via 7×10 mm silicone tubing. A 12 V external power supply powers the pumps through a DC barrel jack distribution setup.

All GPIO pins are initialized in the OFF state (HIGH) at startup, ensuring that pumps cannot activate unintentionally following a power interruption or system restart. This forms a complete cyber-physical system where the Raspberry Pi coordinates estimation, actuation, communication, and storage.

## 3 Data Flow & Communication

---

The system follows a layered IoT communication architecture. The MQTT publish/subscribe protocol is used for all message exchange between the GUI and the Raspberry Pi, owing to its lightweight nature and suitability for event-driven IoT systems.

The system uses five MQTT topics:

- **drfresh/command** — the GUI publishes dispense requests to this topic. Each message specifies the tank (A or B) and the dispense type (**auto** or **manual**). The Raspberry Pi is subscribed and processes incoming commands.
- **drfresh/refill** — the GUI publishes refill events to this topic. Each message specifies the tank and the volume added, allowing the Pi to update the estimated volume in SQLite and write a refill event to InfluxDB.
- **drfresh/status** — the Raspberry Pi publishes operational feedback after each dispensing or refill event, including the tank, type, amount dispensed, and success or failure status.
- **drfresh/alert** — the Raspberry Pi publishes warning and fault notifications, such as low tank level, empty tank, or watchdog-triggered pump closure.
- **drfresh/volume** — the Raspberry Pi publishes the current estimated volume of a tank after every dispense or refill event. This topic allows the GUI to display up-to-date tank levels in real time.

A concrete example of the full data pipeline: the user selects Tank A and automatic dispensing mode from the GUI. The GUI publishes `{"tank": "A", "type": "auto"}` to **drfresh/command**. The Raspberry Pi receives this message, queries SQLite to verify that Tank A has sufficient volume (at least 0.3L), activates GPIO pin 17 to enable the relay and start the pump, waits for the calculated dispense duration, then deactivates the pin. It then updates the volume in SQLite, writes the dispense event and a tank volume snapshot to InfluxDB, computes running analytics such as total volume dispensed and average dispense rate per tank and stores these in InfluxDB as well, and finally publishes feedback to **drfresh/status** and the new volume to **drfresh/volume**. If the remaining volume falls below 0.1L, an alert is also published to **drfresh/alert** and recorded in InfluxDB.

At startup, the Raspberry Pi reads the current tank volumes from SQLite and publishes them to **drfresh/volume**, so the GUI is immediately synchronized with the last known state.

The MQTT client is configured with automatic reconnection logic using an exponential backoff strategy (minimum 1 second, maximum 30 seconds), ensuring the system recovers gracefully from temporary broker or network outages.

## 4 Data Modeling

The system's data model follows the NGSI-LD standard. The JSON example below represents the drink dispenser as a single entity capturing its operational state at a given moment. Attributes include the selected drink type, dispense duration, operational status, and the estimated fill level of each tank expressed as a percentage.

Listing 1: NGSI-LD representation of the DrFresh dispenser entity

```
1 {
2   "id": "urn:ngsi-ld:DrinkDispenser:001",
3   "type": "DrinkDispenser",
4
5   "drinkType": {
6     "type": "Property",
7     "value": "A"
8   },
9
10  "status": {
11    "type": "Property",
12    "value": "completed"
13  },
14
15  "dispenseDuration": {
16    "type": "Property",
17    "value": 5,
18    "unitCode": "SEC"
19  },
20
21  "tankLevelA": {
22    "type": "Property",
23    "value": 65,
24    "unitCode": "P1"
25  },
26
27  "tankLevelB": {
28    "type": "Property",
29    "value": 80,
30    "unitCode": "P1"
31  },
32
33  "observedAt": {
34    "type": "Property",
35    "value": "2026-04-14T10:00:00Z"
36  },
```

```
37
38   "@context": [
39     "https://uri.etsi.org/ngsi-ld/v1/ngsi-ld-core-context.jsonld"
40   ]
41 }
```

The `observedAt` field provides temporal context, ensuring that data updates are time-aware. The inclusion of the standard `@context` enables semantic interoperability with other NGSI-LD-compliant systems and IoT platforms.

## 5 Data Storage Strategy

The system generates event-based data, where each dispensing action, refill, or alert produces a timestamped record. The main data types produced are:

- **Drink Type:** identifies whether Tank A or Tank B was selected.
- **Dispense Type:** indicates whether the operation was automatic or manual.
- **Amount Dispensed:** the volume in litres served during the operation.
- **Status:** confirms whether the dispensing completed successfully or failed.
- **Timestamp:** records the exact date and time of the event.
- **Alert Type:** describes any warning or fault condition (e.g. low tank, empty tank, watchdog closure).
- **Tank Volume:** the estimated remaining liquid in each container, updated after every dispense or refill.

Two storage solutions were evaluated: SQLite and InfluxDB. The final system uses both in a complementary hybrid strategy.

**SQLite** is a lightweight relational database that stores all data in a single local file and requires no separate server process. It runs directly on the Raspberry Pi with minimal resource usage and supports structured SQL queries. In DrFresh, SQLite is used exclusively to store the current estimated volume of each tank. This makes it ideal for the operational control layer: when a dispense command arrives, the Pi queries SQLite synchronously to verify whether enough liquid is available before activating the pump. This read is fast, local, and requires no network dependency.

**InfluxDB** is a time-series database optimized for storing and querying timestamped measurements. It is designed for high-frequency writes and efficient time-based queries, and integrates natively with Grafana for dashboard visualization. In DrFresh, InfluxDB stores

all dispense events, alerts, and tank volume snapshots. In addition, the Raspberry Pi computes analytics and trends after each event — such as total volume dispensed per tank, average dispense rate, dispense frequency over time, and tank depletion trends — and stores these derived metrics in InfluxDB alongside the raw event data. Volume snapshots are written to InfluxDB only when a volume changes, that is after each dispense or refill event, rather than continuously. This keeps write frequency low while still providing a complete historical record of tank level evolution over time.

This hybrid approach separates concerns cleanly: SQLite serves the real-time control logic with minimal overhead, while InfluxDB serves the analytics and visualization layer with a purpose-built time-series store. Grafana reads exclusively from InfluxDB, meaning no direct access to the SQLite file is needed from outside the Pi's control process.

## 6 Processing & System Logic

The system is designed around event-driven triggers and rule-based decision logic. User actions from the GUI act as primary triggers, and the Raspberry Pi applies validation checks before any actuation takes place.

The system supports two dispensing modes. In **automatic mode**, a fixed default volume of 0.3 L is dispensed. When a command is received, the Pi queries SQLite to confirm that the selected tank has at least 0.3 L remaining. If the check passes, GPIO pin 17 (Tank A) or pin 27 (Tank B) is driven LOW to activate the corresponding relay and start the pump. The pump runs for the time required to dispense 0.3 L at the configured flow rate of 0.03 L/s (10 seconds), after which the pin is driven HIGH to stop the pump. The volume in SQLite is then decremented, the dispense event and a tank volume snapshot are written to InfluxDB, and the Pi computes and stores updated analytics — including running totals and depletion trends — in InfluxDB before publishing feedback over MQTT. If the remaining volume after dispensing falls below the low-tank threshold of 0.1 L, an alert is published to `drfresh/alert` and also recorded in InfluxDB.

In **manual mode**, the user initiates dispensing by sending a manual command and keeps the pump running by continuously sending heartbeat messages to `drfresh/command`. The system implements a watchdog mechanism: if no heartbeat is received within a 0.75-second window, the pump is automatically shut off. This protects against scenarios where the user loses connectivity or simply walks away without explicitly stopping the dispense. Additionally, a timeout is calculated at the start of each manual session as `current_volume / FLOW_RATE`. If this timeout elapses, it means the tank would be empty and the pump is closed automatically, preventing the pump from running dry.

**Typical scenario:** a user approaches the Raspberry Pi and views the GUI. They select Tank A and choose automatic dispensing. The Pi checks SQLite and finds Tank A has

0.45 L remaining, which is above the 0.3 L minimum required. The pump activates for 10 seconds and dispenses 0.3 L. The Pi updates SQLite to 0.15 L, writes the dispense event and a tank volume snapshot to InfluxDB, updates the running analytics stored in InfluxDB (such as total volume dispensed from Tank A and the current depletion trend), and publishes the result to `drfresh/status` and the new volume to `drfresh/volume`. Since 0.15 L is above the 0.1 L alert threshold, no alert is triggered. If the user then requests another auto dispense, the Pi detects that only 0.15 L remains (below 0.3 L), rejects the command, publishes a failure status, and notifies the user via `drfresh/alert`.

Refill events are handled through the `drfresh/refill` topic. The user sends the tank identifier and the volume added. The Pi adds this to the current SQLite volume, capped at the maximum tank capacity of 1.5 L, updates SQLite, writes a refill event and a tank volume snapshot to InfluxDB, updates the running analytics in InfluxDB accordingly, and publishes the new volume to `drfresh/volume`.

## 7 Reliability Considerations

The DrFresh system is designed to maintain safe and reliable operation under common failure conditions including communication loss, invalid input, and hardware faults.

If the MQTT broker or network connection becomes temporarily unavailable, the Raspberry Pi continues executing local control logic. Commands already received can still be processed. The MQTT client is configured with automatic reconnection using exponential backoff (1 to 30 seconds), so the connection is restored without manual intervention once the broker is reachable again.

The manual dispensing watchdog is a key reliability mechanism. Because the pump remains active only as long as the client keeps sending heartbeat messages within 0.75-second intervals, any client disconnection — whether due to a crash, network drop, or user inaction — causes the pump to shut off automatically. This prevents accidental over-dispensing and protects both the hardware and the liquid supply.

Each manual session also carries a calculated timeout based on the remaining tank volume divided by the flow rate. This acts as a hard upper bound on how long the pump can run, independently of the watchdog. If the tank would run empty, the pump closes and an alert is recorded.

All GPIO pins are initialized HIGH (pump off) during startup, ensuring pumps cannot activate unintentionally after a power interruption or reboot. Invalid commands received via MQTT are rejected through software validation before any GPIO action is taken. Impossible refill values are also handled gracefully, with the volume capped at the maximum tank capacity.



SQLite is stored locally on the Raspberry Pi, making tank state available even during network outages. InfluxDB, if hosted locally, similarly retains all historical data independently of external connectivity. The separation of operational state (SQLite) from analytics storage (InfluxDB) means a failure in one does not affect the other: if InfluxDB is temporarily unavailable, the control logic continues uninterrupted.

## 8 Data Visualization

---

System data is visualized through a Grafana web-based dashboard that provides real-time monitoring and historical analysis of the dispenser's operation. Grafana reads all data exclusively from InfluxDB, which stores dispense events, alerts, tank volume snapshots, and computed analytics written after each dispense or refill.

The dashboard displays the following information:

- Estimated tank levels for both Tank A and Tank B over time, shown as time-series graphs and current-value gauges.
- Dispensing activity: number of dispenses per tank, volumes dispensed over configurable time windows, and a breakdown by dispense type (auto vs. manual).
- Alert history: a log of all warning events such as low tank levels, empty tank attempts, and watchdog-triggered pump closures, with timestamps.
- Operation status: a record of successful and failed dispense attempts over time.
- Computed analytics and trends: total volume dispensed per tank, average dispense rate, dispense frequency over time, and tank depletion projections.

The dashboard refreshes at intervals of 2–5 seconds, providing near real-time feedback without placing excessive load on the system. Historical data is continuously available for longer-term analysis, such as identifying peak usage periods or tracking how quickly each tank is depleted over a week.

The primary end user of the dashboard is the operator or owner of the dispenser — for example, an office manager — rather than the casual end user. The operator uses the dashboard to monitor system health, detect issues such as frequently low tank levels, and plan maintenance or refill schedules.

Two visualization tools were considered: **Grafana** and **ThingsBoard**. Grafana is a powerful and flexible open-source platform specializing in real-time dashboards and time-series data. It integrates natively with InfluxDB using the Flux query language, supports a wide range of panel types including time-series graphs, gauges, and tables, and can be self-hosted on the Raspberry Pi or a local server with minimal resource usage. Things-

Board, by contrast, offers a complete IoT platform including built-in device management, rule chains, and dashboard capabilities. While this makes it a strong choice for larger deployments with many devices, it introduces significant additional complexity, higher resource requirements, and a steeper configuration overhead that is unnecessary for a two-tank single-device system. Grafana was therefore selected for its simplicity, native InfluxDB support, and strong real-time visualization capabilities.

One example dashboard panel is a dual time-series graph showing the estimated volume of Tank A and Tank B over the past 24 hours. Each dispense event appears as a downward step in the corresponding series, and each refill appears as an upward jump. Threshold markers at 0.1 L indicate the low-tank alert boundary, making it immediately visible when a tank is approaching empty.

## 9 Design Justification

---

The proposed system architecture was selected to balance simplicity, reliability, and ease of implementation while satisfying all project requirements.

The Raspberry Pi was chosen as the central controller because it combines GPIO hardware control with full networking capabilities and Python software support. This allows a single device to manage communication, processing, storage, and actuation without requiring additional microcontrollers or dedicated embedded systems.

MQTT was selected as the communication protocol because it is lightweight, efficient, and well suited to the publish/subscribe messaging pattern used throughout the system. Its support for multiple topics allows clean separation between command, feedback, alert, refill, and volume channels.

The hybrid storage strategy — SQLite for operational state and InfluxDB for event history and analytics — reflects a deliberate separation of concerns. SQLite provides synchronous, low-latency reads for the control logic without requiring a network connection or server process. InfluxDB provides a purpose-built time-series store for raw events, volume snapshots, alerts, and computed analytics such as dispense trends and depletion rates, with native Grafana integration that avoids any need to expose the SQLite file outside the control process. This design would also scale more naturally if the system were extended to more tanks or deployed across multiple dispensers.

Grafana was chosen for visualization due to its powerful dashboard capabilities, native InfluxDB integration, and strong support for both real-time and historical time-series data. Its panel types, including gauges and annotated time-series graphs, are particularly well matched to monitoring tank levels and dispensing activity over time.

Overall, the chosen components satisfy all project requirements while keeping implemen-

tation complexity appropriately low for a two-tank prototype system.

## 10 Implementation Reality Check

---

One practical challenge of the system is that tank levels are estimated rather than directly measured. Since no dedicated liquid level sensors are used, the remaining volume is calculated by decrementing from a known starting value based on cumulative dispensing time and the configured flow rate of 0.03 L/s. This method is cost-effective but can accumulate small errors over time, particularly if the actual pump flow rate varies due to pressure differences or partial blockages.

To reduce estimation drift, the system supports refill commands through the GUI. When a tank is physically replenished, the operator sends the added volume, and the system resets the estimate accordingly. This manual calibration point prevents unbounded error accumulation.

A second challenge is ensuring stable communication between the GUI and the Raspberry Pi. Temporary network interruptions may delay commands or feedback messages. This is addressed by storing operational state locally in SQLite and keeping all core control logic on the Raspberry Pi itself. The MQTT client's automatic reconnection logic further reduces the impact of short outages. In manual mode, the watchdog mechanism provides an additional safety layer: if the connection is lost mid-dispense, the pump closes within 0.75 seconds regardless of the reason for the lost heartbeat.

The layered IoT system architecture of DrFresh is illustrated in Figure 1.

## DrFresh — IoT System Architecture

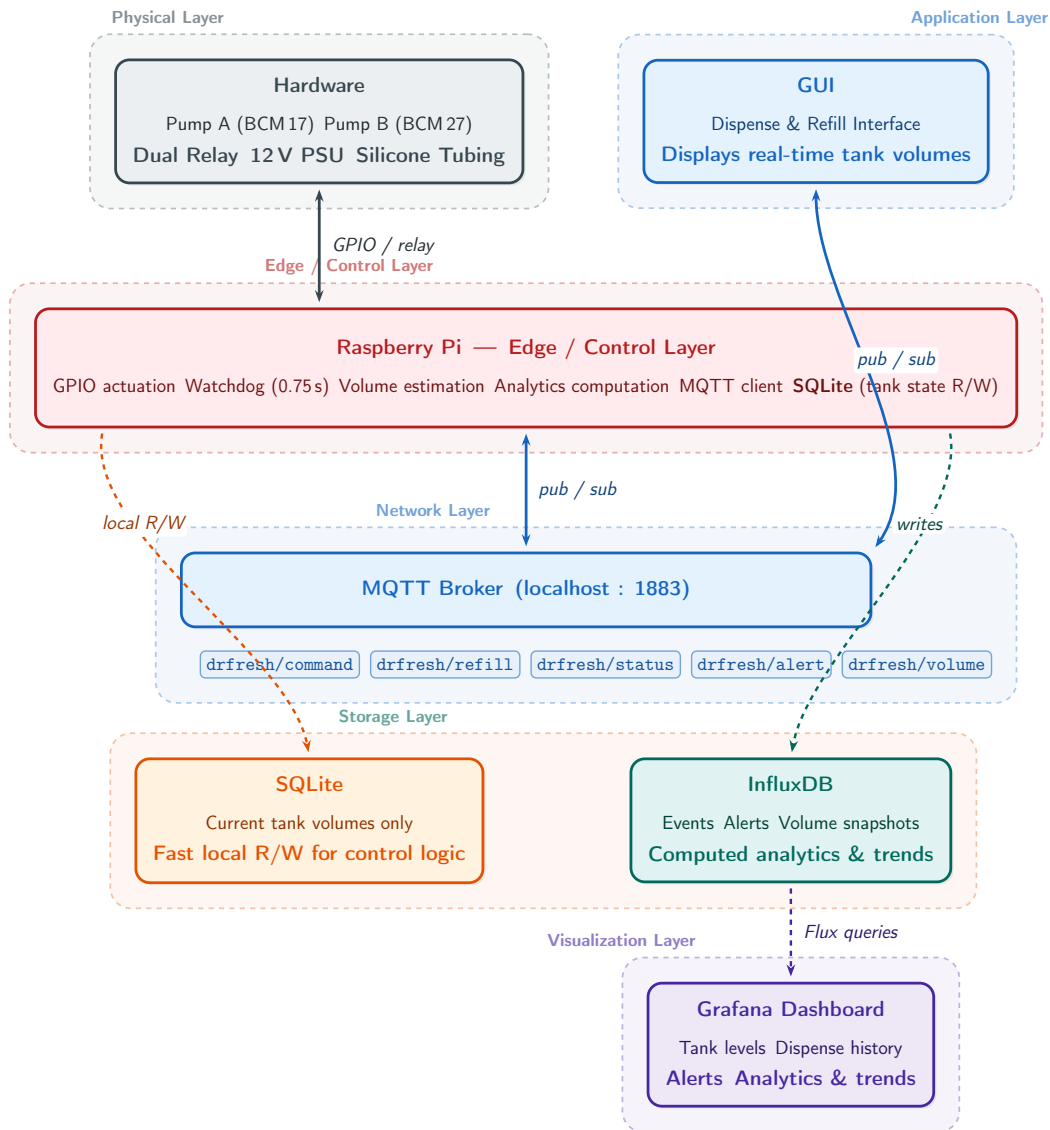


Figure 1: Solid double arrows = bidirectional data flow. Dashed arrows = direct local writes bypassing the broker.

Figure 1: System architecture of the DrFresh smart drink dispenser. Solid arrows indicate bidirectional or directed data flow; dashed arrows indicate direct local writes.